МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений

Студентка гр. 4343 _____ Едина А.

Преподаватель _____ Лисс А.Р.

Санкт-Петербург

2026

Оглавление

Цель работы 3
Задание 4
Исследование функции 5
Вывод 6
ПРИЛОЖЕНИЕ А 10
Исходный код программы 10

Цель работы

Изучение методов приближенного решения нелинейных уравнений. Исследование скорости сходимости и чувствительности методов к погрешностям вычисления функции.

Задание

Найти корень уравнения: $f(x) = \arctan\left(\frac{1}{x}\right) - x^2$ с заданной точностью ε на отрезке. Методы решения:

1. Метод бисекции
2. Метод хорд
3. Метод Ньютона
4. Метод простых итераций

Исследование функции

Исследуемая функция: $f(x) = arctan(\frac{1}{x}) - x^2$

Построим график функции.

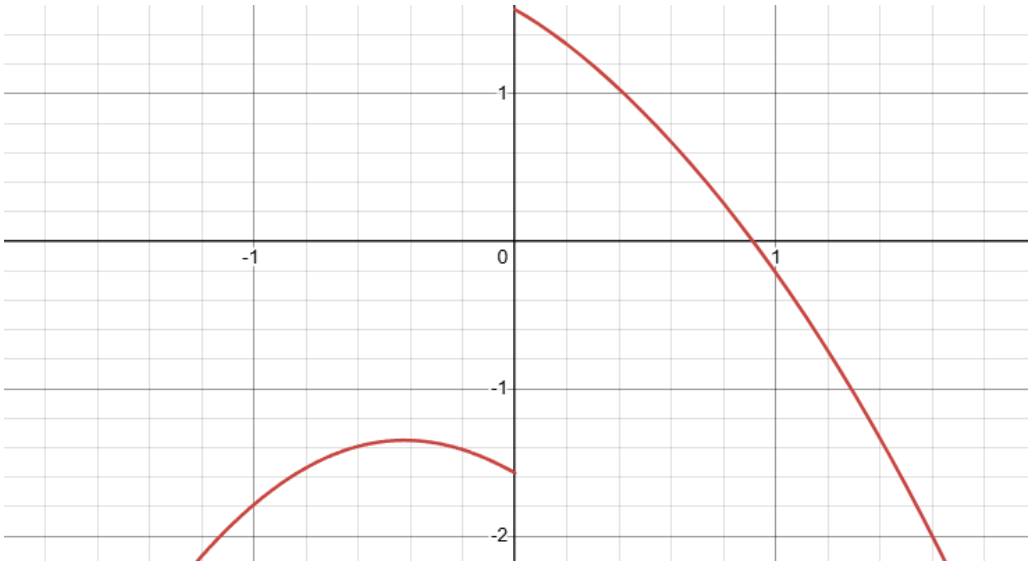

Рисунок 1 - график функции f(x)=arctan(x/1)-x^2

Из графика видно, что подойдет отрезок $[0.5, 1]$.

Докажем, что функция не имеет более 1 корня на этом отрезке.

Вычисление производной.

$$f'(x) = \frac{1}{1+(\frac{1}{x})^2} \cdot \left(-\frac{1}{x^2}\right) - 2x = -\frac{1}{x^2+1} - 2x$$

Докажем, что функция монотонна на промежутке $[0.5, 1]$. Для этого нужно доказать, что первая производная на промежутке, либо строго возрастает, либо строго убывает. Рассмотрим каждое слагаемое первой производной.

$$\frac{1}{x^2+1} > 0$$

$$2x > 0$$

Оба слагаемых отрицательны, следовательно, на интервале $[0.5, 1]$ функция строго убывает.

Найдем вторую производную.

$$f''(x) = \frac{1}{(x^2+1)^2} * 2x - 2$$

Найдем x_0, пусть $x_0 = b,\ b = 1$, проверим условие

$f(x_0)f''(x_0) > 0$

$f(1) = arctan\left(\frac{1}{1}\right) - 1^2 \cong -0.2$

$f''(1) = \frac{1}{(1^2+1)^2} * 2 * 1 - 2 = -1.5$

$f(1)f''(1) = -0.2 * -1.5 = 0.3$

Условие выполняется, возьмем за начальное приближение 1.

Для метода простых итераций для сходимости должно существовать такое вещественное q, что $|\varphi`(x)| \leq q < 1$ для всех принадлежащих $[a, b]$.

Построим график производной.

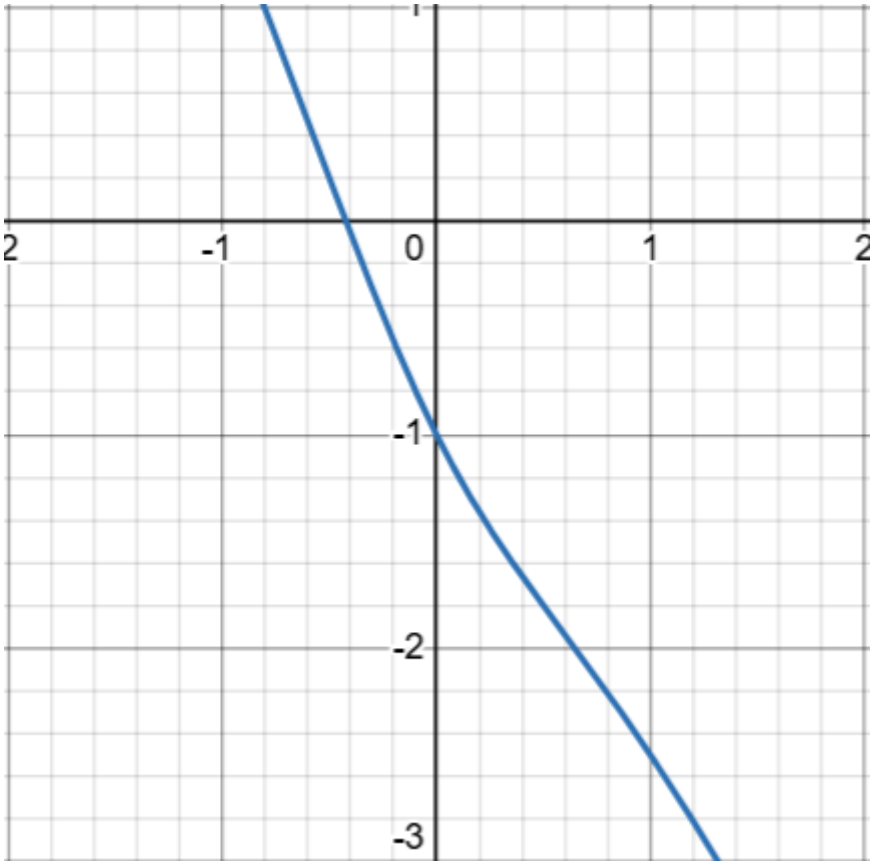

Рисунок 2 - график производной f(x)

Рассмотрим интервал $[0.9, 1]$

$m = minf'(x) = minf'(1) = -2.5$

$M = maxf'(x) = maxf'(0.9) = -2.3$

$\varphi(x) = x - \lambda * f(x)$

$\lambda = \frac{2}{m+M} = \frac{2}{-2.5-2.3} = \frac{2}{-4.8} = -0.42$

$q = \frac{M-m}{M+m} = \frac{-2.3-(-2.5)}{-4.8} = -0.2$

$|q| < 1$

$\varphi(x) = x + 0.42\left(\arctan\left(\frac{1}{x}\right) - x^2\right)$

$\varphi'(x) = 1 + 0.42\left(-\frac{1}{x^2+1} - 2x\right)$

$\varphi'(0.9) = 1 + 0.42\left(-\frac{1}{0.9^2+1} - 2 * 0.9\right) = 0.01$

$\varphi'(1) = 1 + 0.42\left(-\frac{1}{1^2+1} - 2 * 1\right) = -0.05$

Построим график функции $\varphi(x)$.

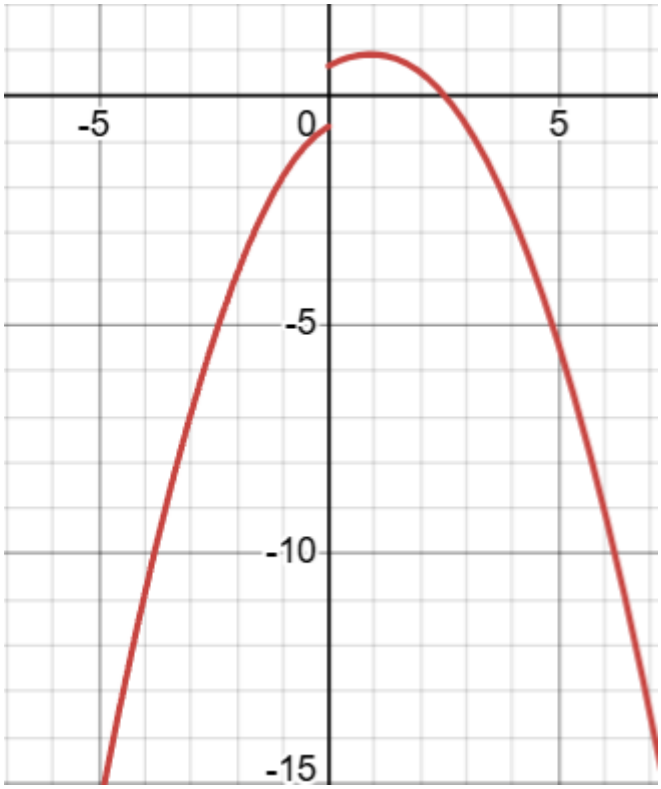

Рисунок 3-график функции φ(x)

Построим график $\varphi'(x)$

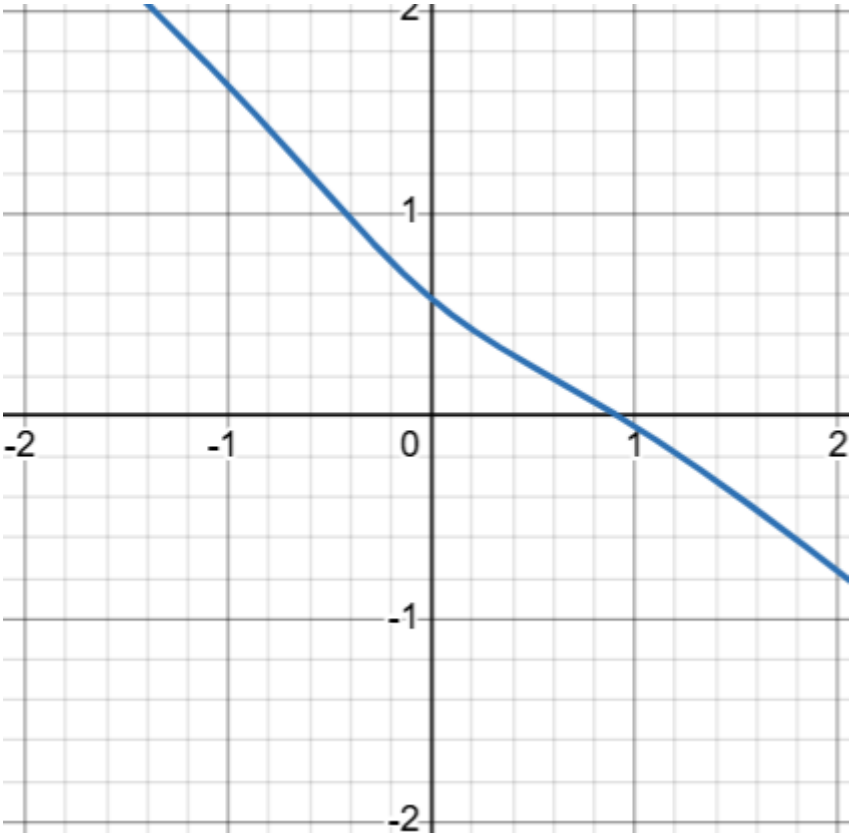

Рисунок 4 - график φ'(x)

Сравнительный анализ методов

Ниже приведен график зависимости числа операций от точности для каждого метода.

Синий – Метод бисекции.

Красный – Метод хорд.

Зеленый – Метод Ньюттона.

Желтый – Метод простых итераций.

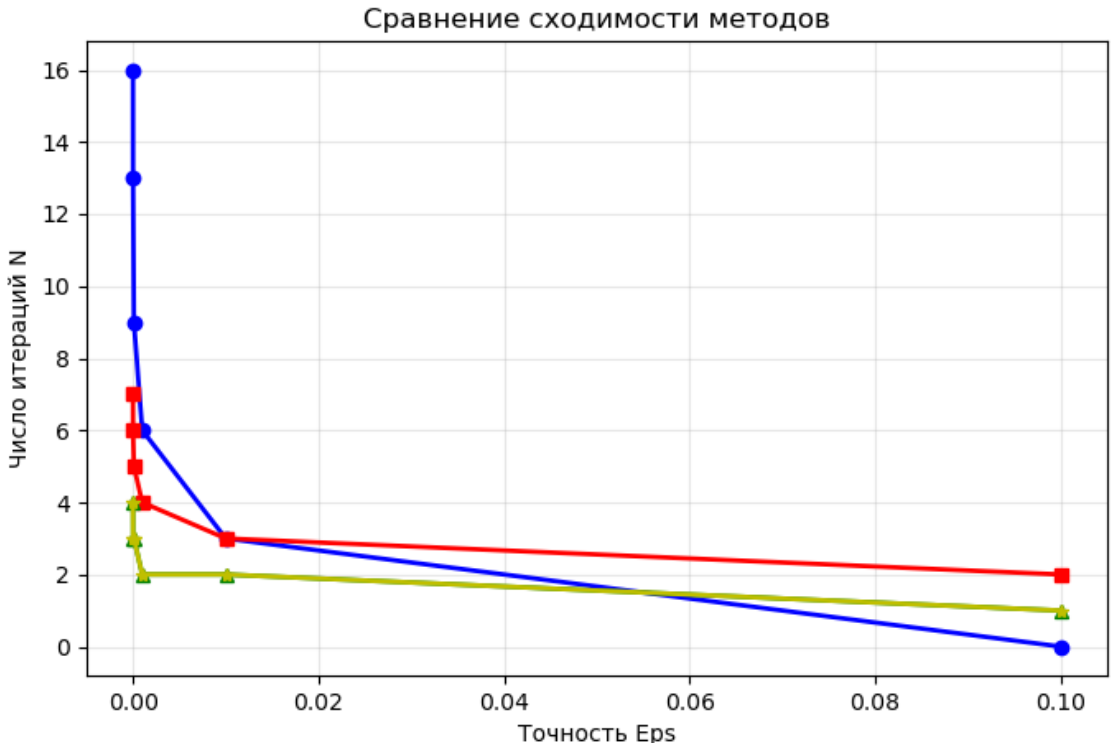

Рисунок 5 - график зависимости числа операций от точности

Далее приведена таблица сравнение количества операций при одинаковой точности для каждого метода.

Таблица 1 - сравнение количесвта итераций

Точность	Бисекции	Хорд	Ньютона	Простых итераций
0.1	0	2	1	1
0.01	3	3	2	2
0.001	6	4	2	3
0.0001	9	5	3	3
0.00001	13	6	3	4
0.000001	16	7	4	4

ПРИЛОЖЕНИЕ А

Исходный код программы

```
#define __NEWTON
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "methods.h"

int UseRound = 0;
double delta = 0;
double Left = 0.9;
double Right = 1;
int N;

double F(double x){
    double res = atan(1/x) - x*x;
    return UseRound ? Round(res, delta) : res;
}

double F1(double x) {
    double res = -1 / (x * x + 1) - 2 * x;
    return UseRound ? Round(res, delta) : res;
}

double F2(double x) {
    return (2.0 * x) / ((x * x + 1.0) * (x * x + 1.0)) - 2.0;
}

double Phi(double x) {
    // φ(x) = x + 0.42 * (arctan(1/x) - x^2)
    double res = x + 0.42 * (atan(1/x) - x*x);
    return UseRound ? Round(res, delta) : res;
}

double Phi1(double x) {
    // Производная φ'(x) = 1 + 0.42 * ( -1/(x^2+1) - 2x )
    double res = 1 + 0.42 * ( -1.0/(x*x + 1.0) - 2.0*x );
    return UseRound ? Round(res, delta) : res;
}

void bisection(){
    printf("\n1 -- to print experemet for N and Eps\n2 -- to choose
your Eps\n3 -- to research sensitivity\n");
    int choice;
    scanf("%d", &choice);
    if(choice == 1){
          double eps_list[] = {0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001};
        int n_eps = sizeof(eps_list) / sizeof(eps_list[0]);
        printf("==================GRID==================\n");
        printf("%-12s | %-12s | %-18s\n", "Eps", "Root", "N");
        printf("_________________________________________\n");
        UseRound = 0;
```

```
        for (size_t i = 0; i < n_eps; i++)
        {
            double Eps = eps_list[i];
            double Root = BISECT(Left, Right, Eps, N);
            printf("%-12.6f | %-12.8f | %-8d\n", Eps, Root, N);
        }
    } else if (choice == 2){
        printf("type your Eps...\n");
        double Eps;
        scanf("%lf", &Eps);
        double Root = BISECT(Left, Right, Eps, N);

        printf("\nYour Eps: %lf\n", Eps);
        printf("Your Root: %5f\n", Root);
        printf("Function: %5f\n", F(Root));
    } else if(choice == 3){
        double Eps = 0.000001;
        double deltas[] = {0.0, 0.001, 0.01, 0.05, 0.1};
        int n_delta = sizeof(deltas) / sizeof(deltas[0]);

       printf("%-10s | %-12s | %-8s | %-10s\n", "Delta", "Root", "N",
"Status");

       printf("==================================================\n");

        for (int i = 0; i < n_delta; i++){
            delta = deltas[i];
            UseRound = (delta > 0) ? 1 : 0;

            double fL = F(Left);
            double fR = F(Right);

            if (fL * fR > 0){
                  printf("%-10.3f | %-12s | %-8s | %-10s\n", delta,
"ERROR", "N/A", "f(L)*f(R)>0");
            } else if (fL == 0.0 || fR == 0.0){
                printf("%-10.3f | %-12s | %-8s | %-10s\n",
                    delta, "ERROR", "N/A", "f(x)=0 на границе");
            }else {
                double Root = BISECT(Left, Right, Eps, N);
                 printf("%-10.3f | %-12.8f | %-8d | %-10s\n", delta,
Root, N, "OK");
            }
        }
    }
    else
    {
        printf("wrong input, bye\n");
        exit(1);
    }
}

void horda(){
    printf("\n1 -- research convergence\n2 -- enter your Esp\n3 --
research for delta\n");
```

```
    int choice;
    double Eps;
    scanf("%d", &choice);

    if (choice == 1){
          double eps_list[] = {0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001};
        int len_list = sizeof(eps_list) / sizeof(eps_list[0]);

        printf("\n%-12s | %-18s | %-8s\n", "Eps", "Корень", "N");
        printf("=============================================\n");

        for (int i = 0; i < len_list; i++){
            double Eps = eps_list[i];
            double Root = HORDA(Left, Right, eps_list[i], N);
            printf("%-12.6f | %-12.8f | %-8d\n", Eps, Root, N);
        }
    } else if (choice == 2){
        printf("enter your Eps...\n");
        scanf("%lf", &Eps);
        UseRound = 0;
        double Root = HORDA(Left, Right, Eps, N);

        printf("\nYour Eps: %lf\n", Eps);
        printf("Your Root: %5f\n", Root);
        printf("Function: %5f\n", F(Root));
    } else if (choice == 3){
        double Eps = 0.000001;
        double deltas[] = {0.0, 0.001, 0.01, 0.05, 0.1};
        int n_delta = sizeof(deltas) / sizeof(deltas[0]);

       printf("%-10s | %-12s | %-8s | %-10s\n", "Delta", "Root", "N",
"Status");

       printf("==================================================\n");

        for (int i = 0; i < n_delta; i++){
            delta = deltas[i];
            UseRound = (delta > 0) ? 1 : 0;

            double fL = F(Left);
            double fR = F(Right);

            if (fL * fR > 0){
                  printf("%-10.3f | %-12s | %-8s | %-10s\n", delta,
"ERROR", "N/A", "f(L)*f(R)>0");
            } else if (fL == 0.0 || fR == 0.0){
                printf("%-10.3f | %-12s | %-8s | %-10s\n",
                    delta, "ERROR", "N/A", "f(x)=0 на границе");
            }else {
                double Root = HORDA(Left, Right, Eps, N);
                 printf("%-10.3f | %-12.8f | %-8d | %-10s\n", delta,
Root, N, "OK");
            }
        }
    }
```

```
    else
    {
        printf("wrong input, bye\n");
        exit(1);
    }
}


// f'(x) = (  1 / ( 1 + (1/x)^2 )  ) · ( -1/x^2 ) - 2x

// f'(x) = (  - 1 / (x^2 + 1)^2   ) - 2x

// f''(x) = (  2x / (x^2 + 1)^2  ) - 2

// f''(x) f(x) > 0
void newton(){
    double Eps, x0, Root;

    printf("\n1 -- research convergence\n2 -- enter your Esp\n3 --
research for delta\n");
    int choice;
    scanf("%d", &choice);

    if (choice == 1){
          double eps_list[] = {0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001};
        int len_list = sizeof(eps_list) / sizeof(eps_list[0]);

        x0 = 1;
        UseRound = 0;

        printf("\nInitial approximination x0 = %.2f\n", x0);
        printf("\n%-12s | %-18s | %-8s\n", "Eps", "Корень", "N");
        printf("=============================================\n");

        for (int i = 0; i < len_list; i++){
            double Eps = eps_list[i];
            double Root = NEWTON(x0, Eps, N);
            printf("%-12.6f | %-12.8f | %-8d\n", Eps, Root, N);
        }
    } else if (choice == 2){
        printf("enter your Eps...\n");
        scanf("%lf", &Eps);
        printf("enter your x0...\n");
        scanf("%lf", &x0);

        double condition = F(x0) * F2(x0);

        if (condition <= 0){
            printf("Initial approximination is a bad. f(x0)·f''(x0) =
%.6f <= 0\n", condition);
        }else {
            printf("condition of convergence is done! f(x0)·f''(x0) =
%.6f", condition);
        }
```

```
        UseRound = 0;
        double Root = NEWTON(x0, Eps, N);

        printf("\nIterations: %d\n", N);
        printf("Your Root: %5f\n", Root);
        printf("Function: %5f\n", F(Root));
    } else if (choice == 3){
        double Eps = 0.000001;
        double deltas[] = {0.0, 0.001, 0.01, 0.05, 0.1, 0.5};
        int n_delta = sizeof(deltas) / sizeof(deltas[0]);
        x0 = 1;

       printf("%-10s | %-12s | %-8s | %-10s\n", "Delta", "Root", "N",
"Status");

       printf("================================================\n");

        for (int i = 0; i < n_delta; i++){
            delta = deltas[i];
            UseRound = (delta > 0) ? 1 : 0;

            double fL = F(Left);
            double fR = F(Right);

            double f1_val = F1(x0);
            if (fabs(f1_val) < 1e-10) {
                  printf("%-10.4f | %-12s | %-8s | %-15s\n", delta,
"ERROR", "N/A", "f'(x0)≈0");
                continue;
            }

            Root = NEWTON(x0, Eps, N);
           printf("%-10.4f | %-12.8f | %-8d | %-15s\n", delta, Root,
N, "OK");
        }
    } else{
        printf("wrong input, bye\n");
        exit(1);
    }
}

double simple_iter(double x0, double eps, int &N){
    double x_prev = x0;
    double x_next;
    N = 0;
    do {
        x_next = Phi(x_prev);
        N++;
        if (fabs(x_next - x_prev) < eps)
            break;
        x_prev = x_next;
        if (N > 10000) {   // запас от зацикливания
           printf("Превышено максимальное число итераций (10000)\n");
            break;
```

```
        }
    } while (1);
    return x_next;
}

void iter(){
    printf("\n1 -- research convergence\n2 -- enter your Esp\n3 --
research for delta\n");
    int choice;
    scanf("%d", &choice);

    if (choice == 1) {
          double eps_list[] = {0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001};
        int len = sizeof(eps_list)/sizeof(eps_list[0]);
        double x0 = 1.0;
        UseRound = 0;

        printf("\n%-12s | %-18s | %-8s\n", "Eps", "Корень", "N");
        printf("=============================================\n");
        for (int i = 0; i < len; i++) {
            double eps = eps_list[i];
            double root = simple_iter(x0, eps, N);
            printf("%-12.6f | %-12.8f | %-8d\n", eps, root, N);
        }
    }
    else if (choice == 2) {
        double eps, x0;
        printf("Enter yor eps...\n ");
        scanf("%lf", &eps);
        printf("Enter your x0 ([0.9, 1]): ");
        scanf("%lf", &x0);

        double dphi = Phi1(x0);
        if (fabs(dphi) >= 1.0)
            printf("warn: |φ'(x0)| = %f >= 1.\n", fabs(dphi));
        else
            printf("info: |φ'(x0)| = %f < 1\n", fabs(dphi));

        UseRound = 0;
        double root = simple_iter(x0, eps, N);
        printf("\nroot: %f\n", root);
        printf("N: %d\n", N);
        printf("value of f(x) = arctan(1/x)-x^2: %f\n", F(root));
    }
    else if (choice == 3) {
        double eps = 0.000001;
        double deltas[] = {0.0, 0.001, 0.01, 0.05, 0.1, 0.5};
        int n_delta = sizeof(deltas)/sizeof(deltas[0]);
        double x0 = 1.0;

        UseRound = 0;
        double exact_root = simple_iter(x0, eps, N);
        int exact_N = N;
```

```
        printf("%-10s | %-12s | %-8s | %-10s\n", "Delta", "Root", "N",
"Status");
        printf("==================================================\n");
         for (int i = 0; i < n_delta; i++) {
             delta = deltas[i];
             UseRound = (delta > 0) ? 1 : 0;
             double root = simple_iter(x0, eps, N);
             double err = fabs(root - exact_root);
             const char* status;

             if (root < 0.8 || root > 1.3)
                 status = "OUT OF RANGE";
             else if (err > 0.0001)
                 status = "LOW ACCURACY";
             else
                 status = "OK";
              printf("%-10.4f | %-12.8f | %-8d | %-12.3e | %-10s\n",
delta, root, N, err, status);
         }
    }
    else{
        printf("wrong input, bye\n");
        exit(1);
    }
}

int main(){
    printf("lab work number 1\n");
    printf("choose method\n");
    printf("1 -- bisection\n2 -- horda\n3 -- newton\n4 -- iter\n");
    int method;
    scanf("%d", &method);

    switch (method)
    {
    case 1:
        bisection();
        break;
    case 2:
        horda();
        break;
    case 3:
        newton();
        break;
    case 4:
        iter();
        break;
    default:
        printf("wrong input, bye\n");
        break;
    }
}
```